



INSTITUTO TECNOLÓGICO SUPERIOR DE LERDO

Nombre del alumno:

Alexander Adonai Molina Rodríguez - 242310131

Nombre del profesor:

Jesús Salas Marín

Nombre de la materia:

Administración y Organización de Datos

Carrera:

Ing. Informática

Fecha:

03/06/2026

Turno:

Matutino

SISTEMA DE CONTROL DE ACCESO (ALMACÉN DE QUÍMICOS)

1. Ecosistema del Proyecto y Competencia Semestral

Este proyecto plantea un ecosistema de software y hardware simulado enfocado en resolver una problemática crítica de seguridad industrial: el control de ingreso a un almacén de productos químicos peligrosos para mitigar riesgos del personal y sustancias activas. La arquitectura se divide de forma modular en tres componentes principales: una capa de configuración y almacenamiento estructurado de usuarios autorizados (`usuarios.json`), un núcleo de control e interactividad en Python (`control_acceso.py`) que valida las credenciales y simula la respuesta física del hardware (apertura de cerradura por 5 segundos o activación de alarma/buzzer), y una interfaz web de monitoreo en tiempo real desarrollada en PHP, HTML, CSS y JavaScript (`index.php`) para desplegar un dashboard con el historial de eventos, contadores y herramientas de búsqueda.

El diseño integral de esta aplicación demuestra de forma directa que adquirí la competencia del semestre: "Evaluar diferentes organizaciones de archivos aplicándolas a situaciones reales". En lugar de implementar de manera automática una base de datos pesada, analizamos el escenario real de una planta industrial que puede adolecer de fallas de red constantes. Al evaluar las alternativas, demostré la capacidad de integrar formatos semiestructurados modernos (JSON) para la gestión jerárquica de perfiles junto con archivos planos secuenciales de texto para la persistencia masiva de bitácoras (logs). Aprendí a contrastar el rendimiento físico de cada organización de archivos en disco, a prever errores en la lectura de formatos mal estructurados y a estructurar flujos lógicos eficientes según las prioridades de persistencia y consulta del negocio.

2. Justificación de Formatos

- **Formato JSON (`usuarios.json`):** Elegido para la base de datos del personal debido a su estructura jerárquica clara basada en pares clave-valor (`id_tarjeta`, `nombre_empleado`, `departamento`, `nivel_seguridad`). Esto permite organizar la información de los empleados de forma muy legible. Científicamente, al procesarse mediante la librería `json` de Python con la función `json.load()`, el archivo se transforma de inmediato en memoria principal en estructuras nativas como listas y diccionarios. Esto facilita una búsqueda veloz y directa sobre los perfiles de los usuarios cada vez que un ID interactúa con el lector, garantizando tiempos de respuesta mínimos.
- **Formato de Texto Plano con Adición (`auditoria.txt`):** Se eligió para el archivo de logs debido a que los registros de eventos de seguridad exigen una escritura constante y un historial inalterable. Al abrir el archivo en Python

usando el modo de escritura 'a' (append), el puntero del sistema operativo se posiciona automáticamente al final del documento. Físicamente, esto significa que cada nuevo intento de acceso se añade directamente sin reescribir ni leer los datos antiguos guardados, logrando una operación de escritura limpia de alta velocidad y protegiendo el historial de pérdidas accidentales de información. Además, al ser un formato plano independiente, no requiere un servidor externo, permitiendo el funcionamiento local del control de acceso si se corta la red de la fábrica.

3. Estimación de Uso y Escalabilidad

El sistema fue estructurado con bloques de control de excepciones try-except en Python para evitar fallos críticos si usuarios.json no existe o si su sintaxis está dañada, lo que eleva su fiabilidad operativa. En condiciones base de prueba (con las transacciones iniciales registradas), cada línea del archivo de auditoría se almacena de forma estructurada ocupando aproximadamente 100 bytes en disco.

¿Qué le ocurre al sistema si recibe una cantidad grande de registros?

- **Operación de Escritura (Python):** La fase de registro de eventos mantiene una complejidad constante. Esto significa que sin importar si auditoria.txt tiene 10 o 500,000 líneas guardadas, la velocidad con la que Python escribe una nueva entrada al final del documento con el modo 'a' seguirá siendo de fracciones de milisegundo, ya que no necesita procesar el histórico en memoria principal.
- **Operación de Lectura y Visualización (PHP):** La fase de monitoreo web se ve directamente afectada de forma lineal. El script en PHP lee la totalidad de la bitácora mediante la función file() para transformarla en un arreglo e invertir el orden visual de las filas con array_reverse(). Si la planta industrial genera un tráfico masivo (por ejemplo, 100,000 intentos de acceso en un mes), el archivo acumulará cerca de 10 MB. Al cargar el panel web, PHP consumirá memoria RAM del servidor de forma proporcional para crear el arreglo y procesar cada línea mediante explode(" - ", \$linea). Ante un volumen excesivamente grande de registros, la página web experimentará retrasos al renderizar la tabla HTML o al ejecutar la función de filtrado en JavaScript en el cliente. Para resolver este comportamiento en el futuro sin perder la independencia del archivo plano, se requerirá implementar una subrutina de paginación de datos para leer únicamente las últimas 50 líneas del documento de texto en lugar del archivo completo.

4. Análisis Costo-Beneficio

Criterio de Evaluación	Costo Técnico / Computacional	Beneficio Operativo
Infraestructura de Datos	Mínimo: No existe costo económico de licencias de software, infraestructura de red de datos compleja ni necesidad de mantenimiento de un servidor de base de datos relacional dinámico.	Muy Alto: El sistema opera localmente de forma autónoma. Esto es ideal para naves industriales aisladas o entornos con conectividad intermitente, reduciendo a cero la dependencia de terceros.
Persistencia y Almacenamiento	Bajo: Los formatos JSON y TXT son archivos de texto sin compresión binaria integrada, lo que podría aumentar un poco el uso de almacenamiento físico a largo plazo en comparación con esquemas binarios cerrados.	Alto: Trazabilidad e independencia total. El archivo auditoria.txt puede abrirse e inspeccionarse con cualquier editor de texto básico, permitiendo auditorías directas sencillas y respaldos manuales rápidos.
Rendimiento de Consultas	Moderado: El dashboard de PHP requiere procesar mediante software los delimitadores de cadena (explode) al leer el archivo secuencial plano, elevando el uso de CPU a gran escala.	Alto: La validación de la tarjeta física por medio de Python se procesa en microsegundos al mapear el JSON en diccionarios, liberando al operador de demoras en los accesos de la entrada.
Seguridad y Control de Acceso	Bajo: El nivel de seguridad se gestiona editando directamente el parámetro entero en el JSON (nivel_seguridad), requiriendo que se cumpla la condición lógica de nivel mayor o igual a 2 para abrir la cerradura.	Muy Alto: Disminución de riesgos en áreas peligrosas al negar el paso de forma automática a perfiles con nivel insuficiente o usuarios desconocidos, alertando visual y sonoramente.

Conclusión

El desarrollo de este sistema de control de accesos reafirma la utilidad práctica de estructurar la información mediante archivos planos y semiestructurados cuando las condiciones reales del entorno exigen una respuesta rápida, bajos costos de operación y un funcionamiento local robusto. Mediante la combinación estratégica de archivos JSON para configuraciones que cambian poco y archivos de texto plano en modo append para registros secuenciales de alta frecuencia, se logró una solución de ingeniería eficiente, trazable y escalable para el entorno de una planta industrial. Esta práctica consolida el criterio para elegir arquitecturas de archivos adecuadas basándose en las restricciones de hardware y la naturaleza del problema de negocio que se busca resolver.